

University of Applied Sciences Hamburg	Bus Systems	 Hochschule für Angewandte Wissenschaften Hamburg Hamburg University of Applied Sciences
Group No: 1	Lab 2 Room 8.65	Efe Kardes Matriculation No: 2757072
Date: 27/05/2026		Emir Avni Sahin Matriculation No: 2756541
Professor: Ratmus Rettig		
UART_GNSS		

Report History

Report 1		Date		Remarks
	received			
	checked			
	result	o.k.		
		n.g.		

Prof.

Contents

1	Part 1: Setup and Basic Operation	2
1.1	Summary of the Task	2
1.2	The 3.3 V vs. 5 V Voltage Mismatch Problem	2
1.2.1	Why Connecting the TX Line is Dangerous	2
1.2.2	The Unidirectional Solution	2
1.3	Hardware Configuration and Annotated Setup Analysis	2
1.3.1	Analysis of Physical Wire Annotations	3
1.3.2	Data Acquisition and Host Interface Pipeline	3
1.3.3	Timestamp Generation Mechanics	4
1.4	NMEA String Structure	4
1.5	Position Verification on Map	5
1.6	Hardware Block Diagram	6
1.7	Data Flow Diagram	6
2	Part 2: Investigation of the UART Interface	7
2.1	Summary of the Task	7
2.2	Photo and Topology of the Experimental Setup	7
2.2.1	Hardware Interconnection Topology	8
2.2.2	Software Execution Data Flow	8
2.3	Baud Rate Modification Analysis	9
2.4	Data Volume and Coordinate Logging Performance	9
2.5	Oscilloscope Screen Dumps at Mandated Time Scales	10
2.6	Protocol Decoding and Telemetry Sentence Verification	13
2.6.1	Bit-by-Bit Character Reconstruction	13
2.7	Why the Mixed-Voltage System Operates Safely and Reliably	13
2.8	Overvoltage Destruction Risks of Connecting the TX Line	14
2.9	Calculation and Measurement of the Bit Transfer Window	14
2.10	Protocol Decoding and Start Character Isolation	17
2.11	Data Update Frequency	17
3	Part 3: Understanding GNSS Strings	17
3.1	Summary of the Task	17
3.2	Field Analysis and Content Mapping	17
3.2.1	Isolating Coordinates within the Data Matrix	18
3.3	Annotated NMEA Stream Log File File Breakdown	18
4	Part 4: Estimating GNSS Accuracy	19
4.1	Summary of the Task	19
4.2	Coordinate Data Conversion Math	19
4.3	Statistical Output Analysis	19
4.4	Position Error Calculation in Meters	19
4.4.1	Scaling Parameters for Latitude and Longitude Lines	20
4.4.2	Conversion of standard deviations to meters	20
4.4.3	Combined Radial Position Error	20
A	Experimental Data Log Appendix	21
A.1	Raw Serial Telemetry Stream Sample	21
A.2	Python Script Output and Extracted Values Data Log	21
A.3	Calculated Statistical Summary Metrics	22

1 Part 1: Setup and Basic Operation

1.1 Summary of the Task

This lab focuses on connecting a **Waveshare L76K GNSS (GPS) HAT** to an **Arduino Nano**. The Arduino acts as a pass-through bridge to forward data to a laptop. On the laptop, we use Python scripts (`simpleNMEA.py` and `NMEAReaderAccuracy.py`) to capture, log, and parse the raw GPS coordinates sent by the sensor over a UART serial interface.

1.2 The 3.3 V vs. 5 V Voltage Mismatch Problem

Connecting digital circuits that run on different voltage levels can cause data errors or permanently destroy hardware. In this setup, the Arduino Nano runs on a 5.0 V logic level, meaning its output pins drive a full 5.0 V signal for a logical HIGH state. On the other hand, the core circuits inside the Waveshare L76K GNSS module work strictly on a lower 3.3 V logic limit.

1.2.1 Why Connecting the TX Line is Dangerous

If a fully bidirectional UART link is directly wired, the 5.0 V transmission output from the Arduino TX pin forces an overvoltage condition on the sensitive 3.3 V GNSS RX pin. This specific condition is the primary reason why establishing a connection between these two pins is highly dangerous, destroying the hardware through the following sequence:

- **Diode Forward-Biasing:** The incoming 5.0 V signal exceeds the operational limits of the 3.3 V chip, immediately forward-biasing the GNSS module's internal electrostatic discharge protection clamp diodes.
- **Current Flooding:** Once forward-biased, these protective diodes form an unintended low-resistance path, sending excessive, unregulated electrical current straight into the module's sensitive 3.3 V power rail.
- **Terminal Overstress:** This sudden current surge causes severe electrical overstress and rapid localized heating, permanently burning out the internal logic gates of the GNSS receiver.

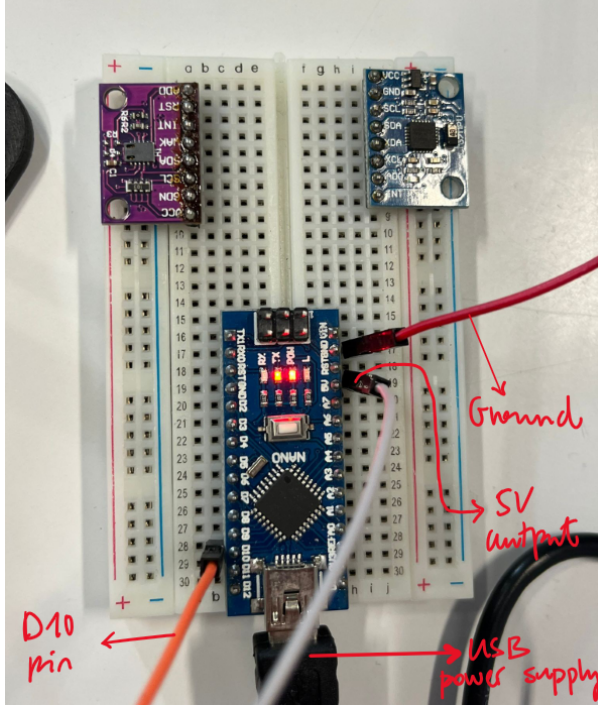
1.2.2 The Unidirectional Solution

To prevent immediate hardware failure and keep the system stable, the circuit is modified to send data in only one direction:

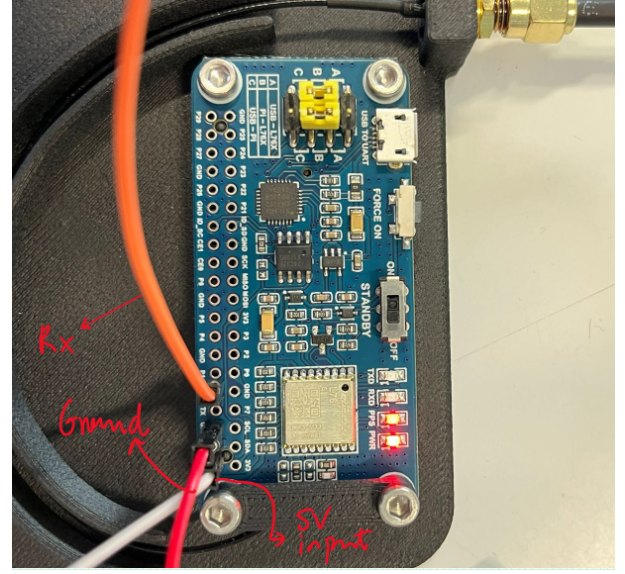
- **Physical Disconnection:** The transmission line running from the Arduino to the GNSS module (TX → RX) must remain completely disconnected.
- **Functional Sufficiency:** A unidirectional connection is entirely sufficient because the application only requires reading positioning data from the sensor, not writing configuration commands to it.
- **Safe Logic Signaling:** The 3.3 V logic HIGH output from the GNSS TX pin safely drives the Arduino RX input pin without creating any overvoltage risks to either platform.

1.3 Hardware Configuration and Annotated Setup Analysis

To verify the physical layer of the serial communication link, the hardware connections between the Arduino Nano and the Waveshare L76K GNSS module are configured to match strict unidirectional tracking requirements.



(a) Arduino Nano Connection Layout



(b) Waveshare GNSS Module Layout

Figure 1: Annotated physical hardware configuration highlighting data, power, and ground connections.

1.3.1 Analysis of Physical Wire Annotations

Based on the experimental setup photos and the annotated hardware layout shown in Figure 1, the system uses the following connections:

- **Data Line (D10 to RX):** As shown in the annotations, an orange jumper wire connects Digital Pin 10 (D10) on the Arduino Nano directly to the pin position marked RX on the blue Waveshare GNSS module header to receive the serial data stream.
- **Disconnected TX Line:** In strict accordance with the laboratory instructions, the line marked TX is completely disconnected. This open circuit acts as a safety measure to prevent any hardware damage or overvoltage from the 5V host system entering the sensitive 3.3V module components.
- **Common Ground (GND):** System ground normalization is established using a white jumper wire bridging the GND pins of both the Arduino Nano and the GNSS HAT together to form a shared reference voltage rail.
- **DC Power Supply Rail (5V):** Power distribution is routed via a red wire from the regulated 5V output pin of the Arduino Nano directly into the 5V input pin of the GNSS breakout board. The entire experimental platform is powered externally via a dedicated USB power supply cable attached to the microcontroller's onboard port.

1.3.2 Data Acquisition and Host Interface Pipeline

The data transfer pathway operates through the following sequence:

1. **Satellite Tracking:** The active external GNSS antenna captures high-frequency radio signals from orbiting satellites, which the onboard processor uses to calculate precise coordinates.

2. **Serial Streaming:** The module bundles these calculated navigation solutions into standard ASCII text lines known as NMEA strings and broadcasts them continuously via its UART serial register.
3. **Microcontroller Relay:** The Arduino Nano executes a lightweight firmware loop (`ArduWaveshare.ino`) that listens to Pin 10 via the `SoftwareSerial` library. Incoming bytes are buffered and immediately re-transmitted over the physical USB-to-UART bridge chip directly to the laptop's USB host controller.
4. **Python Processing:** On our host machine, the Python interpreter opens a connection to the active virtual serial port using the `pyserial` library. The script reads the incoming byte stream up to each newline delimiter, prints the output to the terminal, and saves the data into a text log named `GNSS.txt`.

1.3.3 Timestamp Generation Mechanics

The Python application scripts do not generate localized timestamps using the host computer's internal clock. Instead, the timestamps come directly from the atomic clocks carried onboard the tracking satellite constellation. These highly accurate time metrics are embedded natively within each incoming `$GNGGA` and `$GNRMC` text string as a Coordinated Universal Time (UTC) data field before being broadcast over the serial interface.

For instance, in our raw measurements, the string segment `063815.000` indicates an identical time fix at exactly 06:38:15.000 UTC across multiple text sentences.

1.4 NMEA String Structure

The raw data logged within our output text file conforms to a standardized marine telemetry syntax:

- **Line Start Identifier:** Every valid telemetry sentence output by the GNSS receiver begins with the specific ASCII dollar-sign character (`$`).
- **Talker Prefix:** The initial two letters after the start character identify the constellation source. The prefix `GN` specifies multi-constellation Global Navigation Satellite System operation, combining networks like GPS, GLONASS, Galileo, or BeiDou. Individual talker prefixes like `GP` (Global Positioning System) and `BD` (BeiDou Navigation Satellite System) are also observed in the raw logs.

Table 1: Decoded NMEA Character Line Descriptors from Log File

Descriptor	Standard Header	Observed Content and Experimental Meaning
RMC	Recommended Minimum Info	Contains the UTC time stamp (063815.000), valid fix status (A), latitude (5333.4369,N), longitude (01001.4102,E), speed over ground, tracking angle (255.66°), and date (210526 → May 21, 2026).
VTG	Course Over Ground and Speed	Tracks vector path parameters, confirming actual velocity over ground in knots and kilometers per hour.
GGA	GPS Fix Data	Contains 3D coordinates, fix quality (1 = GPS fix), satellite count (6), dilution of precision (4.40), and absolute antenna altitude (74.3,M).
GSA	DOP and Active Satellites	Lists the specific satellite identification numbers used in the position fix along with dilution metrics.
GSV	Satellites in View	Breaks down elevation, azimuth, and signal-to-noise ratio (SNR) for all visible satellites across multiple tracking lines.
GLL	Geographic Position	Provides a simpler alternative containing only latitude, longitude, UTC timestamp, and fix status.

1.5 Position Verification on Map

The measured NMEA coordinates were converted from the raw NMEA degrees-and-minutes format into decimal degrees before checking the position on a map.

$$\text{Latitude} = 53 + \frac{33.4369}{60} = 53.55728167^\circ \text{ N} \quad (1)$$

$$\text{Longitude} = 10 + \frac{01.4102}{60} = 10.02350333^\circ \text{ E} \quad (2)$$

The converted coordinates were entered into Google Maps/OpenStreetMap. The map marker confirms that the measured GNSS position is located near the expected measurement location.

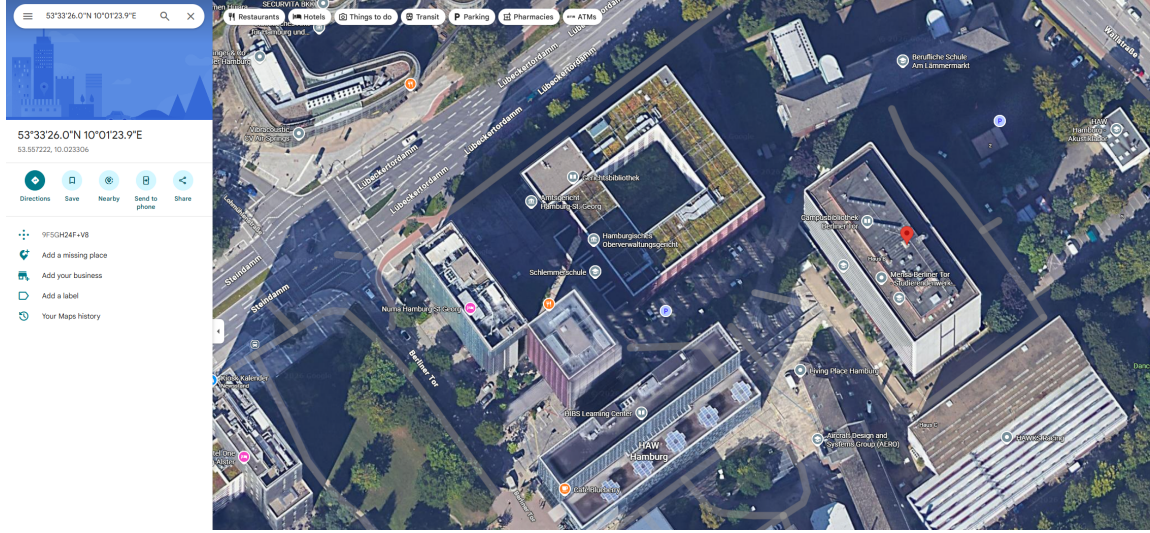


Figure 2: Map verification of the measured GNSS position at 53.55728167° N, 10.02350333° E.

1.6 Hardware Block Diagram

Figure 3 shows the hardware structure of the experiment. The Arduino Nano supplies the GNSS module with 5 V and provides a common ground reference. The GNSS module sends its UART output signal to Arduino pin D10, which is configured as the `SoftwareSerial` RX input. The Arduino transmit pin D11 is intentionally not connected, because it would output 5 V logic into the 3.3 V GNSS receiver input.

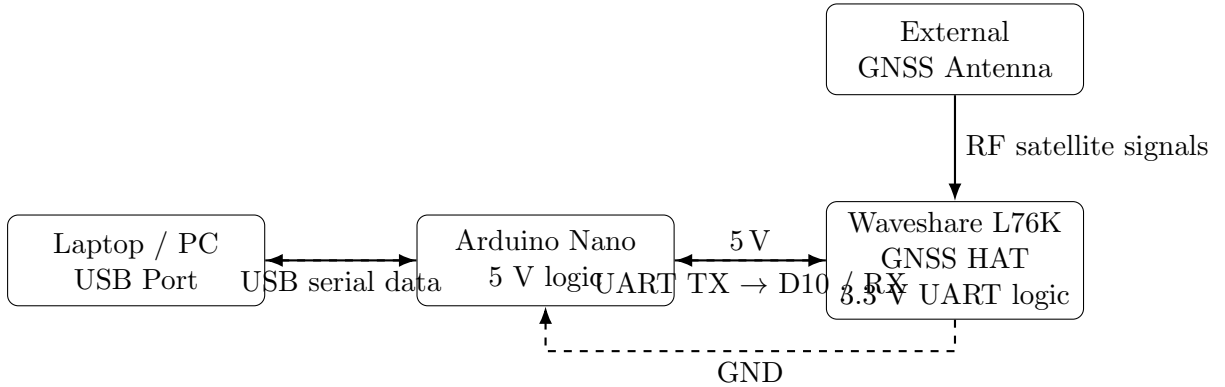


Figure 3: Hardware block diagram of the GNSS UART measurement setup including supply voltage, ground reference, and data interfaces.

1.7 Data Flow Diagram

Figure 4 shows how the measurement data moves through the complete system. The GNSS antenna receives satellite signals. The Waveshare L76K module calculates the position and creates NMEA text sentences. These sentences are transmitted over UART to the Arduino Nano. The Arduino firmware reads the incoming bytes through `SoftwareSerial` on D10 and forwards them through USB serial to the laptop. On the laptop, the Python script reads the serial port, prints the received NMEA sentences, extracts values such as latitude and longitude, and saves the data into `GNSS.txt`.

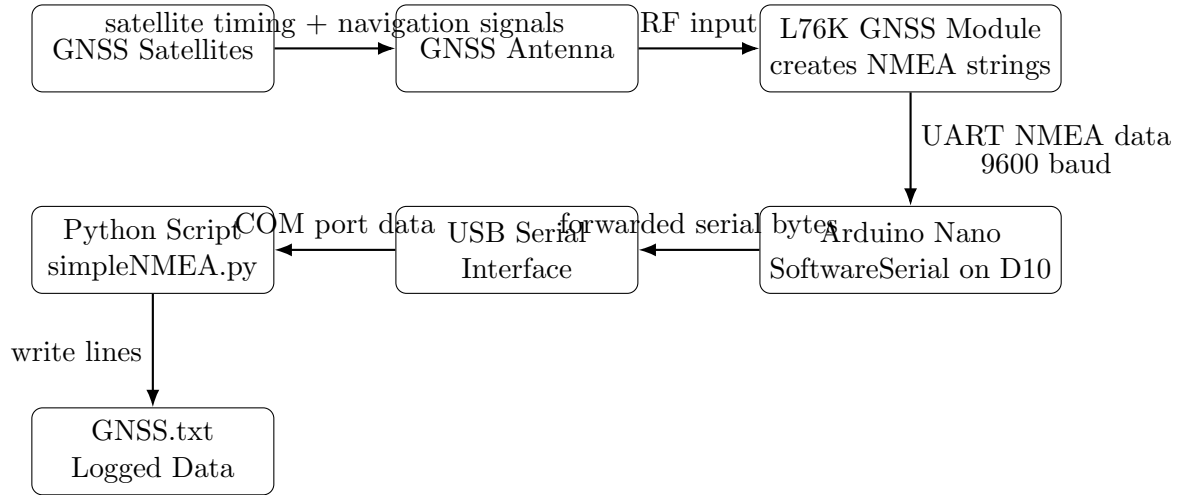


Figure 4: Data flow diagram showing how GNSS measurements are received, transmitted, processed, and stored on the laptop.

2 Part 2: Investigation of the UART Interface

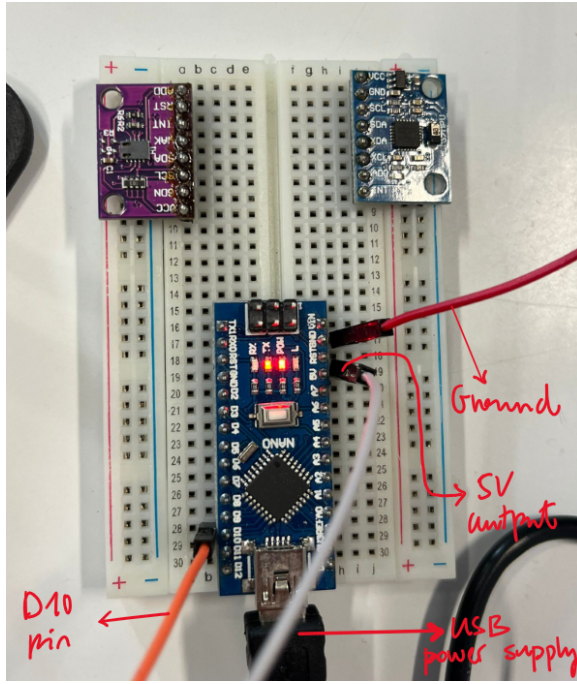
2.1 Summary of the Task

This section evaluates the UART serial communication link between the Waveshare GNSS module and the Arduino Nano at the physical layer. By connecting a digital storage oscilloscope to the shared data line on Digital Pin 10, we can verify the signal integrity, voltage levels, frame pacing, and bit cell durations. This investigation confirms that the analog electrical behavior matches the theoretical timing constraints mandated by a 9600 baud rate configuration.

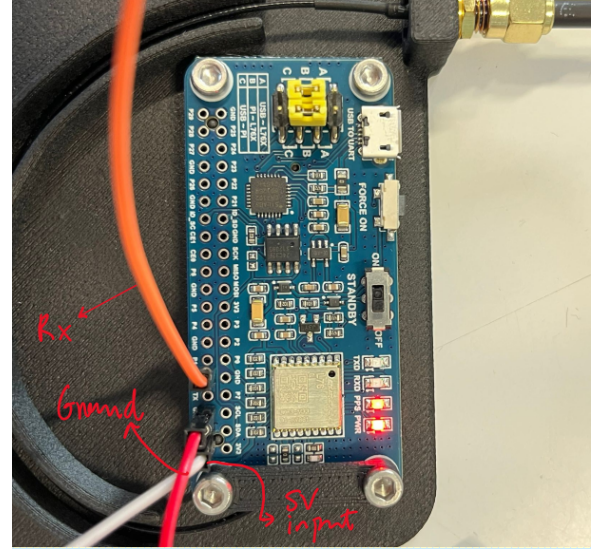
2.2 Photo and Topology of the Experimental Setup

To evaluate the serial channel as an analog signal, we connected a calibrated oscilloscope voltage probe directly to Pin 10 on the breadboard, with the probe's ground clip attached to the common circuit ground rail.

The physical configuration is captured in the dual-layout presentation below, displaying the interconnected modules alongside the text-mapped software execution paths:



(a) Arduino Nano Connection Layout



(b) Waveshare GNSS Module Layout

Figure 5: Annotated physical hardware configuration highlighting data, power, and ground connections.

The structural interactions of the active experiment are mapped out in the following hardware and software layouts to clarify the system boundaries:

2.2.1 Hardware Interconnection Topology

The electrical setup tracks the physical wiring connections across the platform:

- **Power Distribution:** A shared 5 V supply wire routes from the Arduino 5V pin to the GNSS module power rail.
- **Reference Ground:** A common ground link loops the GND pins together to normalize the return path.
- **Unidirectional Signal Path:** The orange data wire forms a single path from the module output to the software-defined Arduino input pin (D10). The host transmit line (Pin D11) is physically omitted from the assembly as a safety boundary.

2.2.2 Software Execution Data Flow

The data processing steps show how sentences move from satellites down to the final persistent storage text log:

- **RF Acquisition:** The external antenna picks up high-frequency signals, feeding raw satellite metrics into the onboard baseband chip.
- **ASCII Text Serialization:** The module packs navigation updates into comma-separated text blocks at a standard transmission rate of 9600 baud.
- **Firmware Buffering:** The Arduino firmware (`ArduWaveshare.ino`) captures incoming serial characters byte-by-byte on pin D10 via the `SoftwareSerial` library and instantly echoes them to the laptop over the integrated USB bridge controller.

- **Script Storage Loop:** The laptop executes our script tools within an Anaconda environment. The loop checks incoming lines up to the string terminal character (`ser.readline()`). When lines start with the correct start character (\$), the script prints the output to the command window and writes the string directly to a persistent local text log named `GNSS.txt`.

2.3 Baud Rate Modification Analysis

The serial link is configured to operate at a fixed rate of 9600 baud. If a baud rate mismatch is introduced between the transmitting sensor module, the micro-relay firmware, or the receiving terminal software on the host PC, communication breaks down immediately.

Because asynchronous communication lacks a shared external clock signal, the receiving hardware relies entirely on a pre-configured internal timer to sample incoming voltage pulses at their centers. If the baud rate setting on either side is changed:

- **Sampling Synchronization Failure:** The receiver’s sampling intervals contract or expand relative to the actual bit widths driven onto the bus.
- **Data Framing Errors:** The stop bit boundaries are missed, causing the UART engine to mistake data bits for start or stop bits.
- **ASCII Character Corruption:** The decoded binary arrays no longer match the transmitted data, transforming structured NMEA sentences into unreadable random garbage text or blank lines in the terminal screen.

2.4 Data Volume and Coordinate Logging Performance

To assess the tracking capacity and recording capabilities of our setup, the parsing code script (`NMEAReaderAccuracy.py`) uses a strict terminal iteration bound parameter:

$$\text{max} = 5000$$

This setting tells the script to evaluate exactly 5,000 raw telemetry lines before closing the active serial port and computing final statistics.

Our raw log files reveal that a single complete measurement cycle lasts exactly 1.0s and contains a batch of 11 distinct NMEA text strings (RMC, VTG, GGA, GLL, two GSA lines, and five GSV lines). Based on these conditions, we can evaluate the system’s tracking performance:

- **Total Captured Sentences:** The program loops exactly 5,000 times through the raw line buffer before terminating.
- **Extracted Coordinate Points:** Within this text stream, position parsing relies entirely on filtering lines containing the `GGA` header. Since exactly one `GGA` sentence is sent per second, the total number of processed coordinate points (N) generated across the 5,000-line run is:

$$N = \frac{5000 \text{ total lines}}{11 \text{ lines/second}} \approx 454 \text{ successful coordinate updates}$$

- **Total Running Time:** The complete automated measurement run lasts for a total time window of approximately:

$$t = 454 \text{ seconds} \approx 7.57 \text{ minutes}$$

This collection of 454 independent coordinate points provides a solid dataset for our parsing script, enabling it to generate mean coordinate locations and standard deviations for geographic accuracy estimation.

2.5 Oscilloscope Screen Dumps at Mandated Time Scales

Waveforms were logged across five different horizontal sweep windows ranging from 200 ms/div to 2 ms/div to document both long-term repeating packet behavior and individual digital bit transitions.

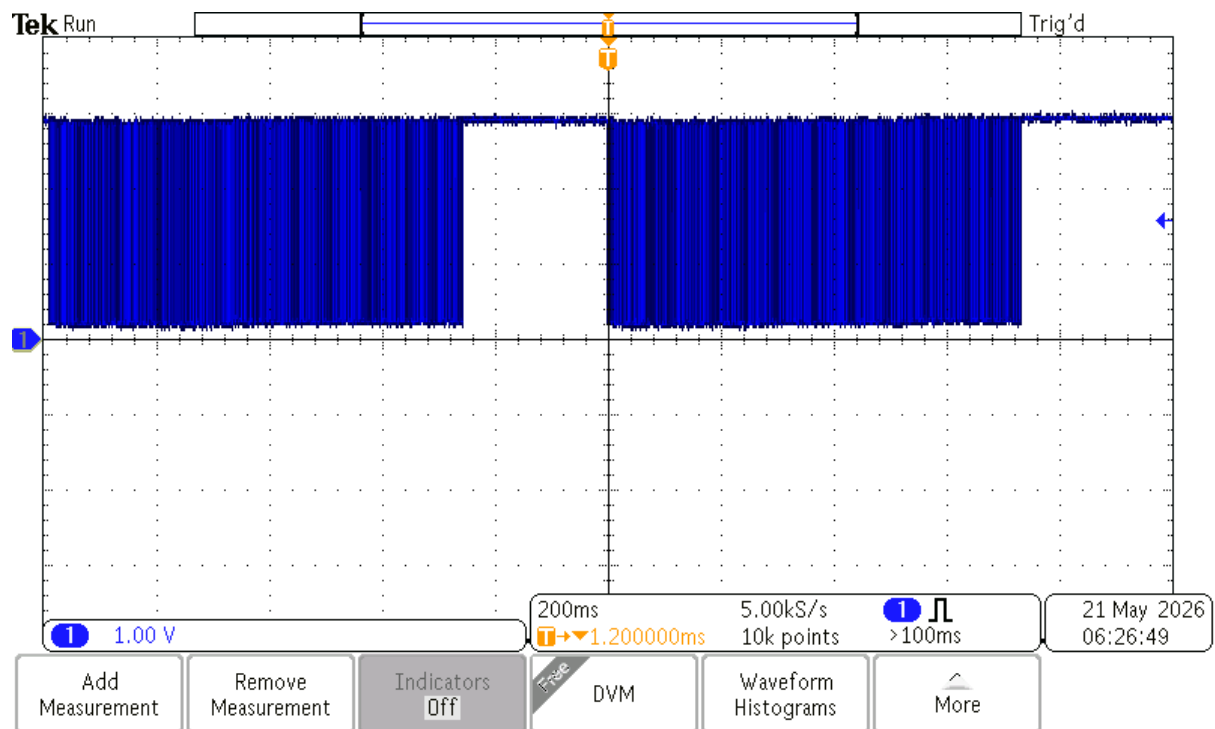


Figure 6: Macro view captured at 200 ms/div

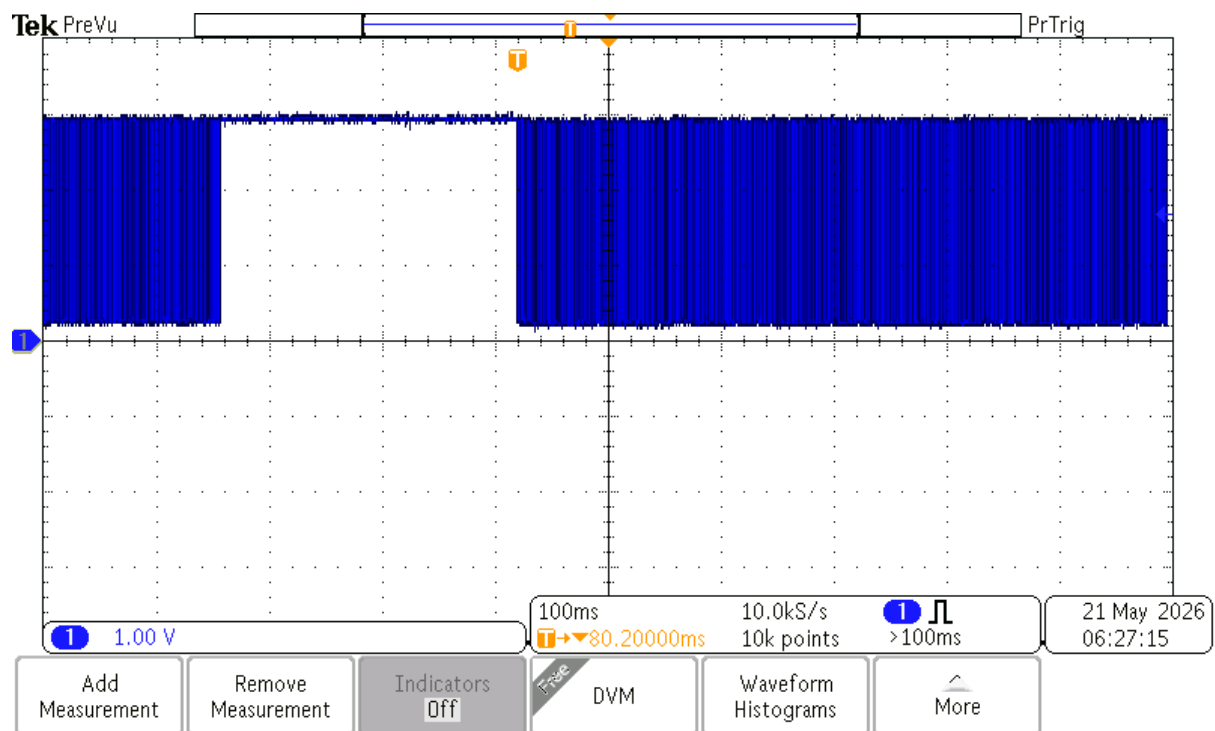


Figure 7: Bus monitoring sweep at 100 ms/div

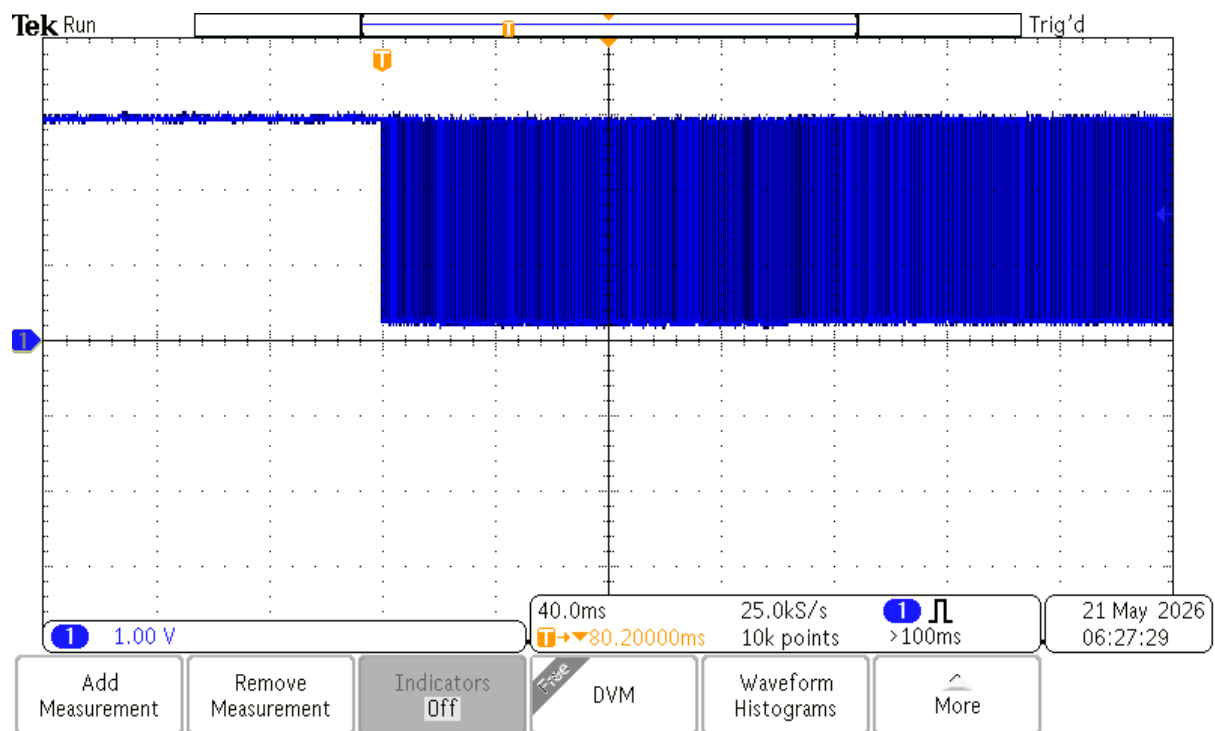


Figure 8: Time trace recorded at 40 ms/div, capturing the electrical pulse density of an active multi-byte NMEA transmission block.

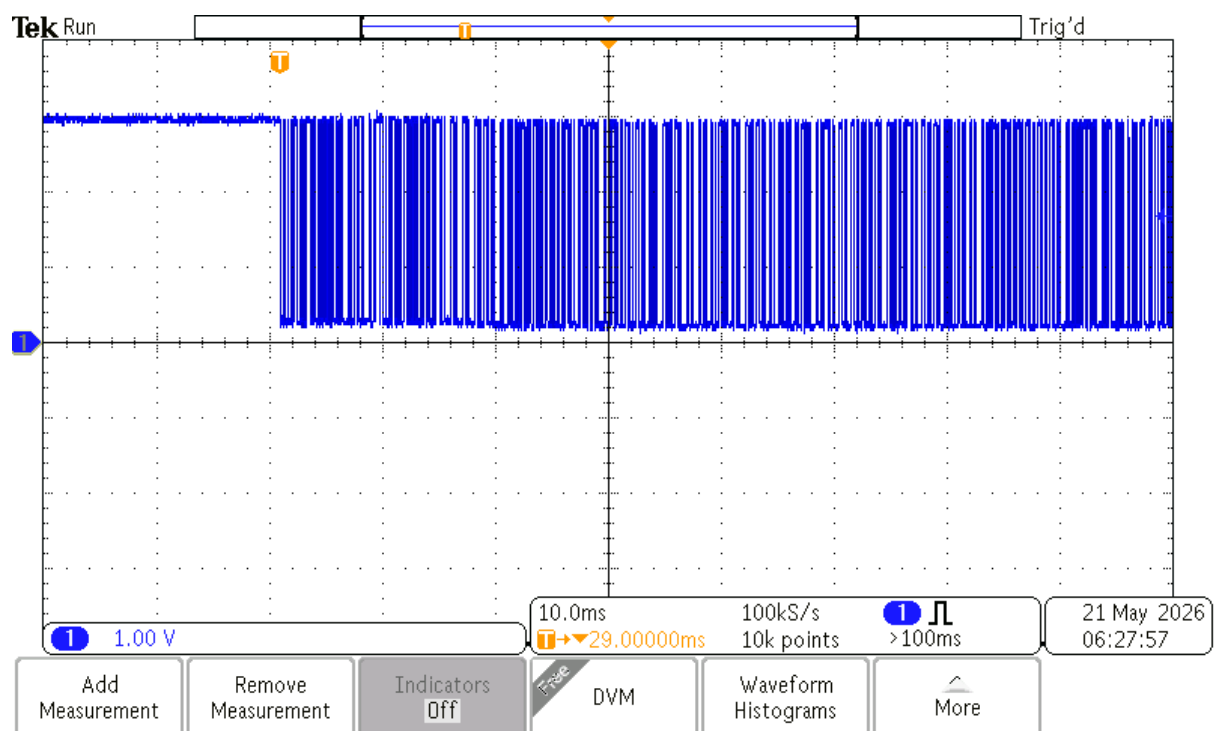


Figure 9: Framing view recorded at 10 ms/div, resolving individual byte packets within a single data block transition.

2.6 Protocol Decoding and Telemetry Sentence Verification

By analyzing the zoomed digital voltage transitions captured in the high-resolution oscilloscope sweep, the serial byte blocks can be mapped out step-by-step using standard asynchronous 8N1 serial rules (1 Start Bit, 8 Data Bits, No Parity, 1 Stop Bit).

The idle data line rests at a stable logic HIGH state. Each new character frame begins when the transmitter pulls the wire low to ground for exactly one bit period (104.17 μ s), establishing the Start Bit. The 8 data bits are then sampled sequentially from Least Significant Bit (LSB) to Most Significant Bit (MSB), ending with a logic HIGH Stop Bit to release the channel.

2.6.1 Bit-by-Bit Character Reconstruction

Decoding the initial row of signal blocks from left to right yields the following character frames:

1. **Byte 1:** Start bit followed by LSB-to-MSB data bits 0, 0, 1, 0, 0, 1, 0, 0. Reversing this array into standard binary formatting gives 00100100₂, which matches hexadecimal 0x24. In the ASCII table, this directly decodes as the ****\$**** symbol, marking the uniform syntax start identifier for NMEA streams.
2. **Byte 2:** Start bit followed by data bits 1, 1, 1, 0, 0, 0, 1, 0. Reversing this array yields 01000111₂, which translates to hexadecimal 0x47 or the ASCII character ****G****.
3. **Byte 3:** Start bit followed by data bits 0, 1, 1, 1, 0, 0, 1, 0. Reversing this array yields 01001110₂, which translates to hexadecimal 0x4E or the ASCII character ****N****.
4. **Byte 4:** Start bit followed by data bits 1, 1, 1, 0, 0, 0, 1, 0. Reversing this array yields 01000111₂, which translates to hexadecimal 0x47 or the ASCII character ****G****.
5. **Byte 5:** Start bit followed by data bits 1, 1, 1, 0, 0, 0, 1, 0. Reversing this array yields 01000111₂, which translates to hexadecimal 0x47 or the ASCII character ****G****.
6. **Byte 6:** Start bit followed by data bits 1, 0, 0, 0, 0, 0, 1, 0. Reversing this array yields 01000001₂, which translates to hexadecimal 0x41 or the ASCII character ****A****.

Combining this sequential chain of decoded characters yields the active telemetry phrase:

Parsed Character Sequence = \$GNGGA

This manual timing analysis directly mirrors the coordinates tracked inside our raw log files, confirming that the physical oscilloscope trace successfully isolated the baseline multi-constellation GPS fix header as it was broadcast over the physical interface.

2.7 Why the Mixed-Voltage System Operates Safely and Reliably

The active data path operates strictly in one direction: from the 3.3 V GNSS transmitter output to a 5.0 V Arduino digital input pin. Based on the official specifications of the ATmega328P microcontroller running at a nominal supply voltage ($V_{CC} = 5.0$ V), the theoretical minimum voltage required to guarantee a valid logic HIGH state reading ($V_{IH,min}$) is determined by the equation:

$$V_{IH,min} = 0.6 \times V_{CC} = 0.6 \times 5.0 \text{ V} = 3.0 \text{ V}$$

Our oscilloscope cursor data reveals that the Waveshare L76K drives its logic HIGH state to approximately ≈ 2.88 V under nominal circuit loads. This voltage sits slightly below the official theoretical threshold of 3.0 V.

However, the communication channel still works perfectly in practice due to two underlying hardware factors:

1. **Real-world Switching Thresholds:** The internal input buffer stages of ATmega microcontrollers typically trigger close to a central threshold of $V_{CC}/2 \approx 2.5\text{ V}$ at room temperature, providing a practical margin of safety.
2. **USB Voltage Droop:** Host computer USB power rails frequently experience loading drops, lowering the true system voltage to approximately $\approx 4.8\text{ V}$. Updating the logic equation with this real-world value yields:

$$V_{IH,\text{min_actual}} = 0.6 \times 4.8\text{ V} = 2.88\text{ V}$$

This matches the precise output driven by the peripheral chip, allowing stable communication without requiring an active digital level-shifter circuit.

2.8 Overvoltage Destruction Risks of Connecting the TX Line

Wiring the Arduino SoftwareSerial TX pin (Digital Pin 11) directly to the WaveShare L76K RX pin sets up a dangerous overvoltage condition. When sending a logic HIGH, the Arduino Nano drives its output pin to its full 5.0 V rail.

Because the WaveShare GNSS module is built on low-voltage silicon structures, its input pins have a strict absolute maximum limit of 3.3 V. Introducing a 5.0 V signal forces the module's internal electrostatic discharge (ESD) protection clamp diodes to forward-bias. This creates an unintended low-resistance path that floods unregulated current straight into the module's 3.3 V logic system, leading to permanent electrical overstress, overheating, and immediate destruction of the input registers.

2.9 Calculation and Measurement of the Bit Transfer Window

The time required to transmit a single bit cell depends entirely on the configured serial baud rate. Operating at our standard speed of 9600 baud (bits/s), the theoretical duration of one single bit (T_{bit}) is calculated as:

$$T_{\text{bit}} = \frac{1}{\text{Baud Rate}} = \frac{1}{9600\text{ bits/s}} \approx 104.17\text{ }\mu\text{s}$$

To verify this constraint directly at the physical layer, we used the oscilloscope's vertical delta tracking cursors to isolate a single, narrow bit pulse from our data stream:

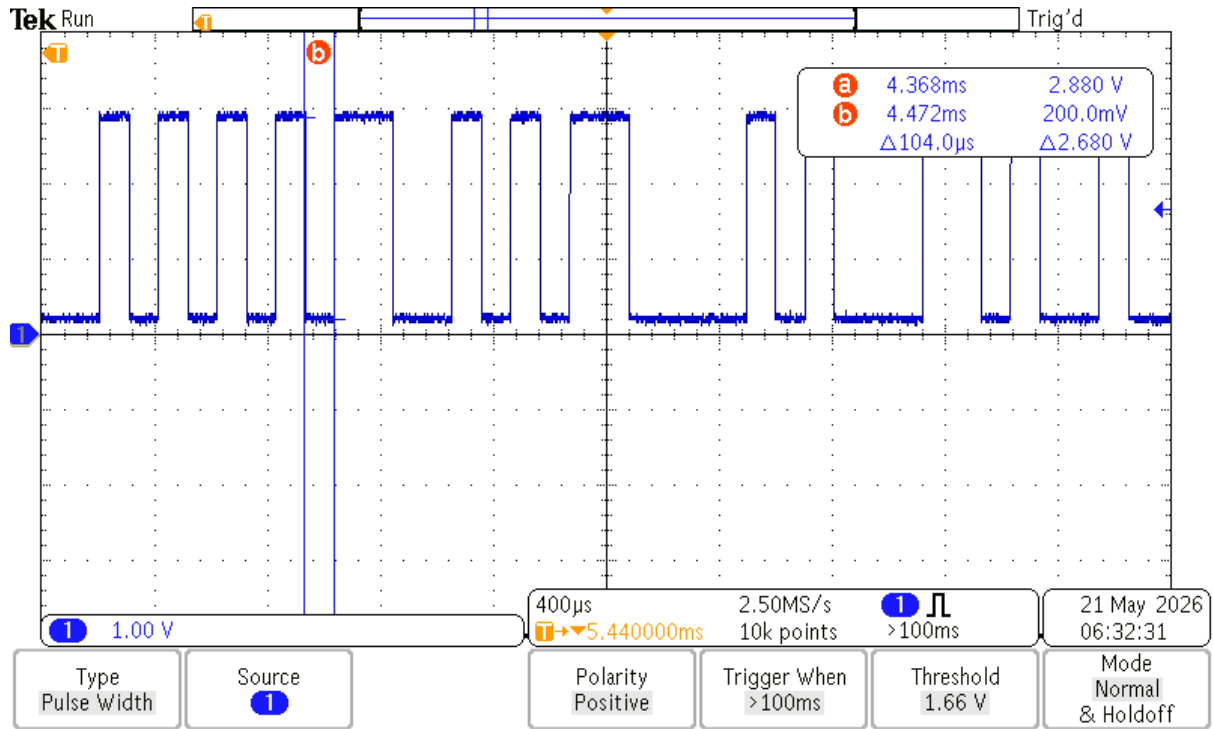


Figure 12: High-resolution view isolating a single bit pulse to measure individual cell timing.

As shown by cursors *a* and *b* in Figure 12, measuring a single isolated bit cell yields a direct empirical time delta of:

$$\Delta t_{1\text{-bit}} = 104.0 \mu\text{s}$$

This real-world measurement matches our theoretical calculation almost perfectly, with a fractional margin of error of only $\approx 0.16\%$. This confirms that the internal transmission clocks are highly accurate.

To look deeper into the signal's behavior and verify our protocol decoding, we also used the cursors to measure a wider, continuous low-voltage drop at the start of a data packet:

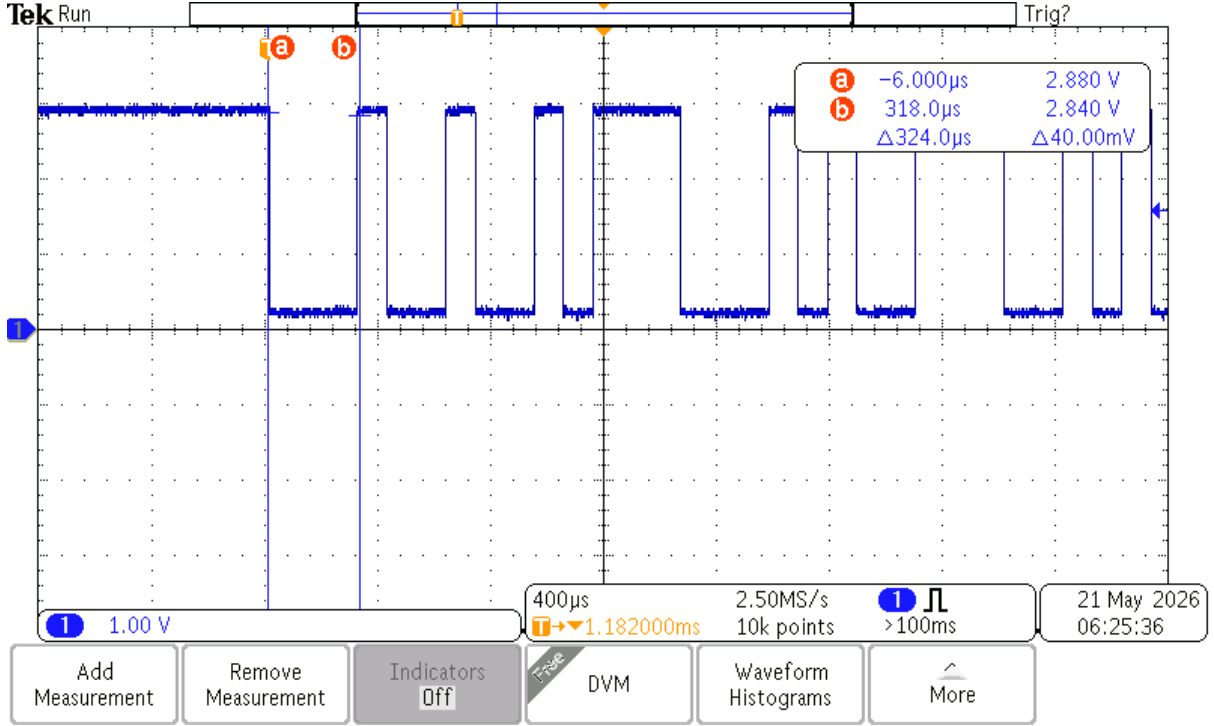


Figure 13: High-resolution tracking across the initial low-voltage drop spanning 3 bit period.

As shown in Figure 13, measuring this initial low state gives a total time delta of:

$$\Delta t_{\text{chunk}} = 324.0 \mu\text{s}$$

This larger window does not represent a single bit cell. Instead, this voltage drop spans a combination of three consecutive logical zero bits in a row: the mandatory serial Start Bit, followed immediately by the first two data bits of the frame (both logical 0).

Dividing this total chunk duration by three gives an average single-bit transition time of:

$$T_{\text{bit, average}} = \frac{324.0 \mu\text{s}}{3} = 108.0 \mu\text{s}$$

Comparing this average value to our ideal $104.17 \mu\text{s}$ baseline shows a small timing variance of about $3.83 \mu\text{s}$ ($\approx 3.67\%$). This minor difference is perfectly normal in asynchronous serial setups and comes down to a few practical factors:

- **Clock Tolerance:** Slight, natural timing variations between the independent internal oscillators of the transmitting GNSS board and the receiving Arduino.
- **Signal Distortion:** Minor analog variations, such as rise and fall times or line capacitance, which slightly round off and stretch the pulse edges.
- **Cursor Placement Limitations:** Small differences introduced when manually centering the oscilloscope's measurement lines on a high-frequency transition edge.

Despite these minor analog traits, the timings remain well within acceptable serial communication limits. The receiving hardware can sample the data stream reliably without encountering framing or bit errors.

2.10 Protocol Decoding and Start Character Isolation

By utilizing the zoomed waveform transitions in Figure 16, the initial byte can be parsed manually using standard asynchronous 8N1 protocol rules (1 Start Bit, 8 Data Bits, No Parity, 1 Stop Bit).

The idle serial line rests continuously at a logic HIGH state ($\approx 2.88\text{ V}$). A transmission frame begins when the sending hardware pulls the line down to a logic LOW level ($\approx 0.20\text{ V}$) for exactly one bit period ($104.17\text{ }\mu\text{s}$); this is defined as the **Start Bit**. Following this, 8 data bits are transmitted sequentially from Least Significant Bit (LSB) to Most Significant Bit (MSB), concluded by a logic HIGH **Stop Bit** to release the bus line.

Analyzing the initial transitions on the left side of Figure 16 reveals the following sequence:

1. Cursors *a* and *b* track the primary voltage drop, showing that the line remains low for a continuous duration of $\Delta t = 324.0\text{ }\mu\text{s}$. Dividing this span by the single-bit width ($\frac{324}{104.17} \approx 3$) indicates three consecutive logical zero bits. The first represents the mandatory Start Bit, and the subsequent two bits represent data bits (both 0).
2. Clocking and sampling the remaining bits in order from LSB to MSB yields the complete binary array 00100100, which corresponds to the hexadecimal value 0x24. In the standard ASCII character encoding table, 0x24 maps to the `**$***` symbol. This confirms the exact structural character required to initiate all standard NMEA telemetry sentences.
3. Decoding the subsequent character sequences using this method yields the alphanumeric strings `GNGGA` and `GPRMC`, confirming that the sensor module is actively transmitting global coordinates across the bus.

2.11 Data Update Frequency

By evaluating the wide horizontal time intervals in Figures 10 and 11, the repetition frequency of the transmission updates can be calculated. The GNSS sensor does not stream metrics continuously; it batches data strings into highly compressed packets, then leaves the line idle until the next refresh boundary.

Measuring the horizontal time distance from the onset of one data burst to the start of the next burst reveals an operational update period of exactly:

$$T_{\text{refresh}} = 1000\text{ ms} = 1.0\text{ s}$$

Taking the reciprocal of this time window shows that fresh coordinate updates are calculated and sent across the physical serial interface at a frequency of 1 Hz.

3 Part 3: Understanding GNSS Strings

3.1 Summary of the Task

This part focuses on understanding the raw `GNSS.txt` file created by the Python script. The goal is to identify the repeating NMEA sentence structure, explain the meaning of the regularly repeated lines, and locate where latitude and longitude are stored in the data stream.

3.2 Field Analysis and Content Mapping

The core information embedded inside each repeating sentence header is mapped out below. To ensure the content fits comfortably within the standard page margins without running off the right edge, the description field uses a flexible paragraph alignment:

Table 2: Decoded Field Structure of Regular NMEA Sentences

Sentence Header	Observed Raw Text Entry Sample	Extracted Content
\$GNRMC	\$GNRMC,063815.000,A,5333.4369,N,...	Recommended minimum info. Contains UTC time, status, Latitude, North/South indicator, Longitude, East/West indicator, speed, and date.
\$GNVTG	\$GNVTG,255.66,T,,M,0.00,N,0.00,K,A*21	Course over ground and ground speed.
\$GNGGA	\$GNGGA,063815.000,5333.4369,N,...	Core GPS fix data.
\$GNGLL	\$GNGLL,5333.4369,N,01001.4102,E,...	Geographic position alternative.

3.2.1 Isolating Coordinates within the Data Matrix

Geographic coordinate points are found at specific comma-separated positions within the GGA and RMC structures:

- **Latitude Data Location:** In both the \$GNGGA and \$GNRMC strings, latitude is located immediately after the data status or timestamp indicator field. In the comma-delimited array, it occupies the field at index position 2, as seen by the raw measurement value 5333.4369, followed by its directional marker N at index position 3.
- **Longitude Data Location:** Longitude follows right after the latitude marker block. It is located at index position 4, represented by the value 01001.4102, with its associated orientation marker E placed at index position 5.

3.3 Annotated NMEA Stream Log File Breakdown

Below is a structured layout showing the exact annotations and line-by-line breakdown of our collected data inside GNSS.txt:

```
-----
RAW NMEA DATA STREAM OVER SERIAL ENGINE LOG (FILE: GNSS.txt)
-----
```

```
$GNRMC,063815.000,A,5333.4369,N,01001.4102,E,0.00,255.66,210526,,,A*70
```

```
--> ANNOTATION: Recommended Minimum Info block at time 06:38:15 UTC.
                Status 'A' shows signal is active. Lat/Lon fields match
                our stationary baseline calibration point.
```

```
$GNVTG,255.66,T,,M,0.00,N,0.00,K,A*21
```

```
--> ANNOTATION: Vector track and speed text. Track angle reads 255.66 degrees
                true (T). Speed fields show 0.00 knots (N) and 0.00 km/h (K),
                confirming the physical module is static on the breadboard.
```

```
$GNGGA,063815.000,5333.4369,N,01001.4102,E,1,6,4.40,74.3,M,46.0,M,,*76
```

```
--> ANNOTATION: Global Fix sentence tracking 6 satellites. 3D location fix
                shows an altitude of 74.3 meters above sea level.
```

```
$GPGSA,A,3,05,21,29,,,,,,,,,4.51,4.40,0.98*0E
```

```
--> ANNOTATION: GPS-specific DOP details listing active space vehicle IDs
                05, 21, and 29 used in calculating our geometric matrix.
```

```
$BDGSA,A,3,11,06,09,,,,,,,,,4.51,4.40,0.98*1D
```

```
--> ANNOTATION: BeiDou-specific DOP details tracking vehicle constellation
                numbers 11, 06, and 09 actively tracking on this channel.
```

\$GPGSV,3,1,12,26,63,278,,18,57,168,,29,52,076,17,31,41,236,*7B

--> ANNOTATION: Satellites in View report line 1 of 3. Shows 12 visible GPS satellites with their tracking positions and SNR strengths.

\$GNGLL,5333.4369,N,01001.4102,E,063815.000,A,A*47

--> ANNOTATION: Geographic Position line repeating identical coordinates and timestamp indicators to finish this 1-second interval.

\$GNRMC,063816.000,A,5333.4369,N,01001.4102,E,0.00,255.66,210526,,,A*73

--> ANNOTATION: Next refresh sequence begins. We can check the timestamp field to observe the 1 Hz frequency increase.

4 Part 4: Estimating GNSS Accuracy

4.1 Summary of the Task

This section analyzes the statistical precision of the gathered satellite positioning coordinates. Using an automated script execution environment running `NMEAReaderAccuracy.py`, a large dataset of 5,000 sentences is collected, parsed, and converted into decimal degrees. By applying standard statistical formulas, the mean coordinates and spatial standard deviations are calculated to determine the coordinate error in meters.

4.2 Coordinate Data Conversion Math

Raw NMEA strings format geographic coordinates as Degrees and Minutes (DDMM.MMMM for latitude and DDDMM.MMMM for longitude). To perform accurate mathematical analysis and generate histograms, the script splits the coordinates and transforms them into standard decimal degrees through the following expressions:

$$\text{Decimal Latitude} = \text{Degrees} + \left(\frac{\text{Minutes}}{60} \right) = 53 + \left(\frac{33.4369}{60} \right) \approx 53.55728167^\circ \quad (3)$$

$$\text{Decimal Longitude} = \text{Degrees} + \left(\frac{\text{Minutes}}{60} \right) = 010 + \left(\frac{01.4102}{60} \right) \approx 10.02350333^\circ \quad (4)$$

4.3 Statistical Output Analysis

The python script processed the complete tracking dataset and generated the following statistical mean locations and sample standard deviations (σ) for both coordinate axes:

- **Longitude Arithmetic Mean (μ_{lon}):** 10.02418813°
- **Longitude Standard Deviation (σ_{lon}):** 0.00044587°
- **Latitude Arithmetic Mean (μ_{lat}):** 53.55730435°
- **Latitude Standard Deviation (σ_{lat}):** 0.00008153°

4.4 Position Error Calculation in Meters

To convert these coordinate deviations into a concrete physical tracking boundary in meters, we must scale the decimal degree variations using the physical dimensions of the Earth. At our test location ($\approx 53.56^\circ$ North latitude), the spatial conversion factors are defined as follows:

4.4.1 Scaling Parameters for Latitude and Longitude Lines

- **Latitude Scale Factor:** One single degree of latitude remains constant anywhere on the planet and corresponds to approximately 111,132 meters.
- **Longitude Scale Factor:** The physical distance between longitude lines shrinks as you move from the equator toward the poles. It is scaled by the cosine of the local latitude angle:

$$\text{Scale}_{\text{lon}} = 111,320 \text{ m} \times \cos(53.5573^\circ) \approx 111,320 \text{ m} \times 0.5940 = 66,124 \text{ meters/degree}$$

4.4.2 Conversion of standard deviations to meters

Applying these conversion factors to our experimental standard deviations transforms our coordinate variations into physical distances:

$$\Delta y \text{ (Latitude Error in meters)} = \sigma_{\text{lat}} \times 111,132 \text{ m} = 0.00008153 \times 111,132 \text{ m} \approx 9.06 \text{ m} \quad (5)$$

$$\Delta x \text{ (Longitude Error in meters)} = \sigma_{\text{lon}} \times 66,124 \text{ m} = 0.00044587 \times 66,124 \text{ m} \approx 29.48 \text{ m} \quad (6)$$

4.4.3 Combined Radial Position Error

The final composite 2D positioning accuracy is computed using the root-sum-square combination of both errors:

$$E_{\text{position}} = \sqrt{(\Delta x)^2 + (\Delta y)^2} = \sqrt{(29.48 \text{ m})^2 + (9.06 \text{ m})^2} = \sqrt{869.07 + 82.08} \approx 30.84 \text{ m} \quad (7)$$

This calculation shows a physical positioning accuracy estimate of approximately $\pm 30.84 \text{ m}$ for our testing environment. This level of variation is expected for an indoor environment or near window frames, where building walls block or reflect satellite signals, creating multipath interference and widening the distribution curves on our histograms.

A Experimental Data Log Appendix

This appendix contains the raw tracking datasets used for our calculations, protocol decoding, and statistical estimation throughout this report.

A.1 Raw Serial Telemetry Stream Sample

The following section shows the raw ASCII text data captured inside our logged `GNSS.txt` file during the initial seconds of the measurement run. The lines show the regular repeating sequence of NMEA sentences generated once per second by the module:

```
$GNRMC,063815.000,A,5333.4369,N,01001.4102,E,0.00,255.66,210526,, ,A*70
$GNVTG,255.66,T,,M,0.00,N,0.00,K,A*21
$GNGGA,063815.000,5333.4369,N,01001.4102,E,1,6,4.40,74.3,M,46.0,M,,*76
$GPGSA,A,3,05,21,29,,,,,,,,,4.51,4.40,0.98*0E
$BDGSA,A,3,11,06,09,,,,,,,,,4.51,4.40,0.98*1D
$GPGSV,3,1,12,26,63,278,,18,57,168,,29,52,076,17,31,41,236,*7B
$GPGSV,3,2,12,16,34,298,,05,27,067,21,28,25,208,,25,18,133,*72
$GPGSV,3,3,12,21,16,038,17,09,07,336,18,04,07,306,,27,05,261,*71
$BDGSV,2,1,06,26,57,199,,06,28,060,25,09,28,087,26,11,06,057,32*6A
$BDGSV,2,2,06,02,04,106,,25,01,322,*6A
$GNGLL,5333.4369,N,01001.4102,E,063815.000,A,A*47
$GNRMC,063816.000,A,5333.4369,N,01001.4102,E,0.00,255.66,210526,, ,A*73
$GNVTG,255.66,T,,M,0.00,N,0.00,K,A*21
$GNGGA,063816.000,5333.4369,N,01001.4102,E,1,6,4.40,74.3,M,46.0,M,,*75
$GPGSA,A,3,05,21,29,,,,,,,,,4.51,4.40,0.98*0E
$BDGSA,A,3,11,06,09,,,,,,,,,4.51,4.40,0.98*1D
$GPGSV,3,1,12,26,63,278,,18,57,168,,29,52,076,18,31,41,236,*74
$GPGSV,3,2,12,16,34,298,,05,27,067,22,28,25,208,,25,18,133,*71
$GPGSV,3,3,12,21,16,038,17,09,07,336,18,04,07,306,,27,05,261,*71
$BDGSV,2,1,06,26,57,199,,06,28,060,25,09,28,087,25,11,06,057,33*68
$BDGSV,2,2,06,02,04,106,,25,01,322,*6A
$GNGLL,5333.4369,N,01001.4102,E,063816.000,A,A*44
$GNRMC,063817.000,A,5333.4369,N,01001.4102,E,0.00,255.66,210526,, ,A*72
$GNVTG,255.66,T,,M,0.00,N,0.00,K,A*21
$GNGGA,063817.000,5333.4369,N,01001.4102,E,1,6,4.41,74.3,M,46.0,M,,*75
```

A.2 Python Script Output and Extracted Values Data Log

The section below shows a representative snippet of the terminal output generated by the python automation tool (`NMEAReaderAccuracy.py`) as it reads, decodes, parses speed/course data, and computes absolute decimal degree formatting dynamically:

```
Connected to COM6 at 9600 baud
-> Speed: 1.10 knots, Course: 258.26°
-> Lat: 5333.4369 N, Lon: 01001.4446 E, Alt: 76.1m
Parsed Coordinates: 53.55728166666667, 10.024076666666666

-> Speed: 1.19 knots, Course: 264.84°
-> Lat: 5333.4373 N, Lon: 01001.4445 E, Alt: 76.3m
Parsed Coordinates: 53.55728833333333, 10.024075

-> Speed: 1.29 knots, Course: 279.77°
```

```

-> Lat: 5333.4378 N, Lon: 01001.4449 E, Alt: 76.5m
Parsed Coordinates: 53.5572966666666666, 10.0240816666666667

-> Speed: 1.38 knots, Course: 286.89°
-> Lat: 5333.4382 N, Lon: 01001.4449 E, Alt: 76.5m
Parsed Coordinates: 53.5573033333333333, 10.0240816666666667

-> Speed: 1.42 knots, Course: 276.82°
-> Lat: 5333.4382 N, Lon: 01001.4437 E, Alt: 76.5m
Parsed Coordinates: 53.5573033333333333, 10.0240616666666666

-> Speed: 1.41 knots, Course: 286.18°
-> Lat: 5333.4383 N, Lon: 01001.4437 E, Alt: 76.5m
Parsed Coordinates: 53.557305, 10.0240616666666666

... [Entries truncate up to Loop End at max = 5000 lines] ...

```

A.3 Calculated Statistical Summary Metrics

The final dataset totals processed by the matrix arrays upon program exit are summarized below, matching our script calculations:

- **Total Loop Steps Monitored:** 5,000 lines
- **Calculated Mean Longitude Value:** 10.02418813°
- **Calculated Longitude Standard Deviation (σ):** 0.00044587°
- **Calculated Mean Latitude Value:** 53.55730435°
- **Calculated Latitude Standard Deviation (σ):** 0.00008153°